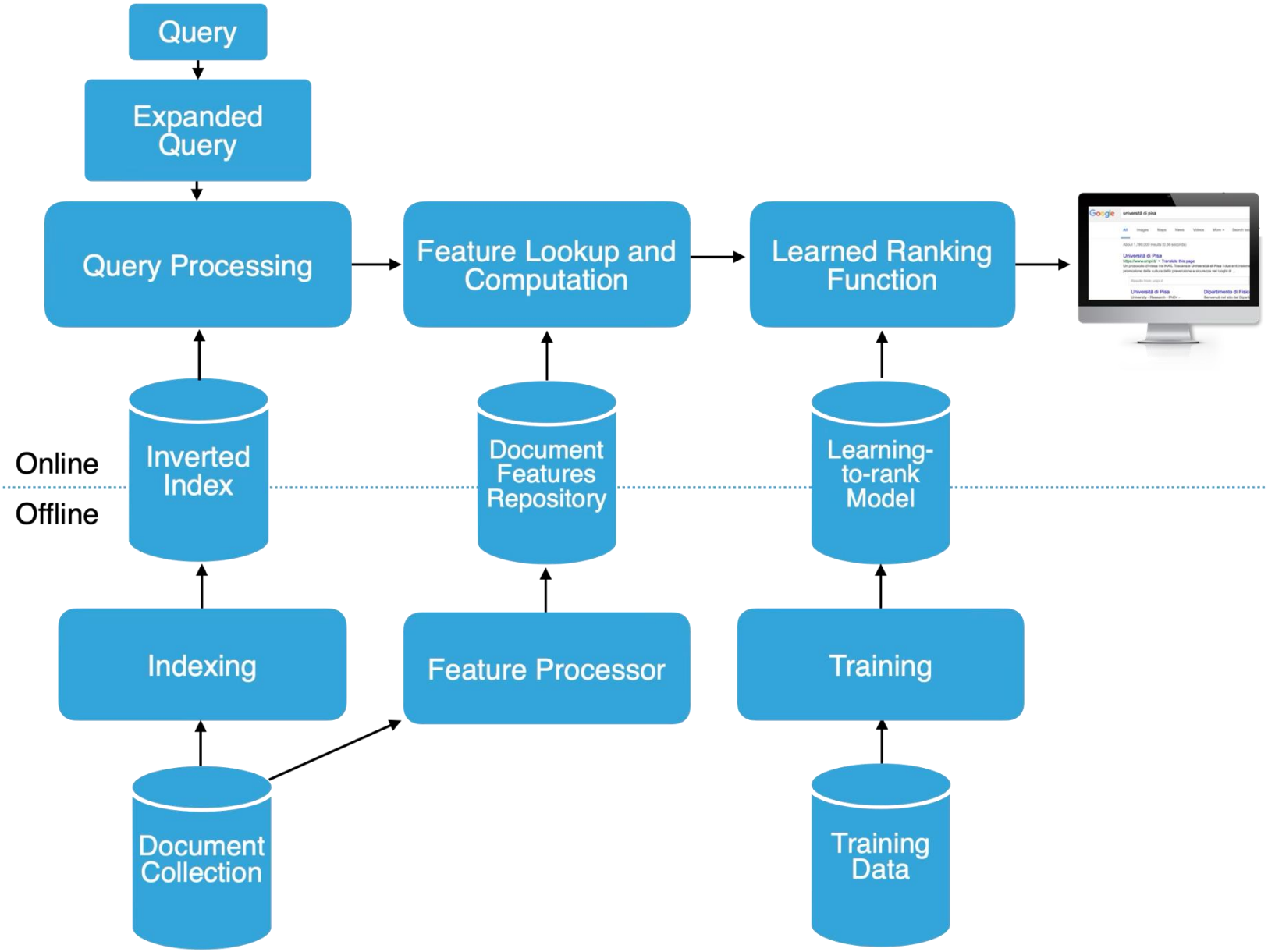
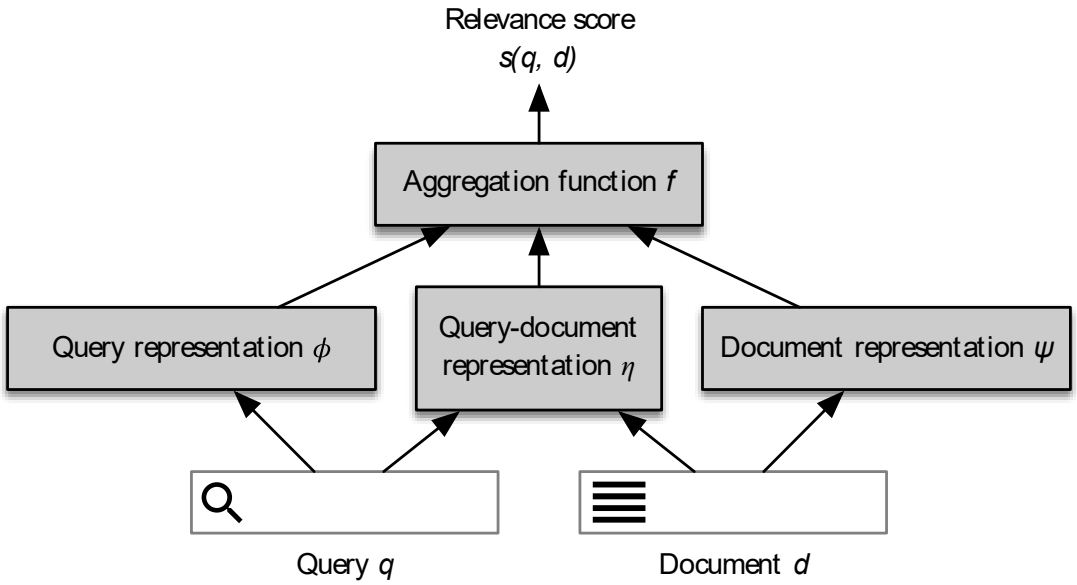
LEARNING TO RANK


Query
Expanded Query
Query Processing
Feature Lookup and Computation
Learned Ranking Function
Online
Offline
Inverted Index
Document Features Repository
Learning-to-rank Model
Indexing
Feature Processor
Training
Document Collection
Training Data

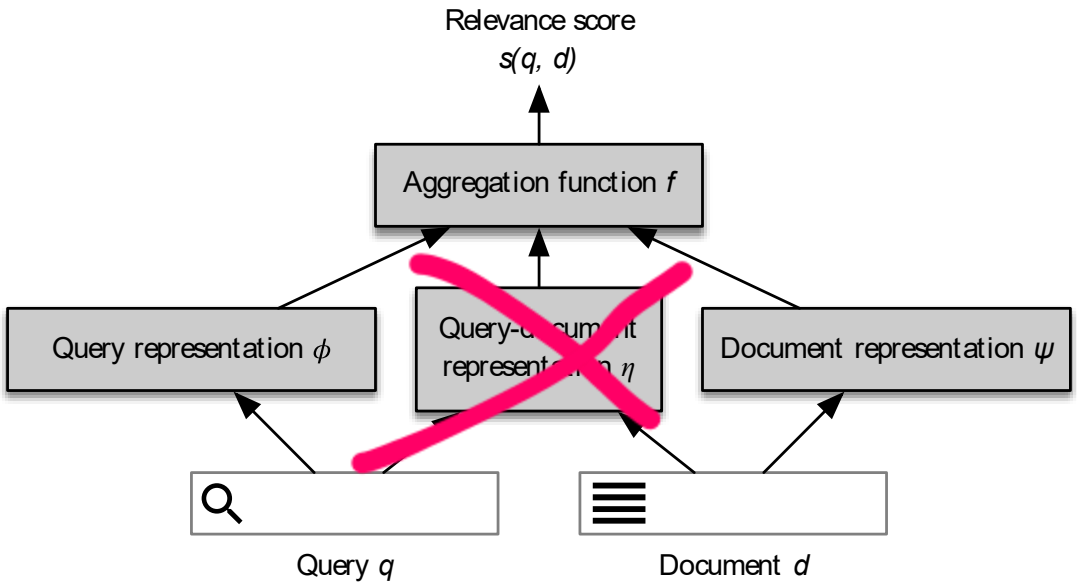
General ranking functions



- Representation-based decomposition of a ranking function. Adapted from [Guo et al. 2020]

[J. Guo, Y. Fan, L. Pang, L. Yang, Q. Ai, H. Zamani, C. Wu, W. B. Croft, and X. Cheng. 2020. A deep look into neural ranking models for information retrieval. Information Processing & Management, 57(6): 1–20.]

BOW ranking functions



- Query and Documents are represented as **sparse BOW vectors**
- No query-document features
- Example of function *f*
  - *cosine, BM25*
- Sparse representations stored in inverted indexes, i.e., the backbone of commercial Web search engine

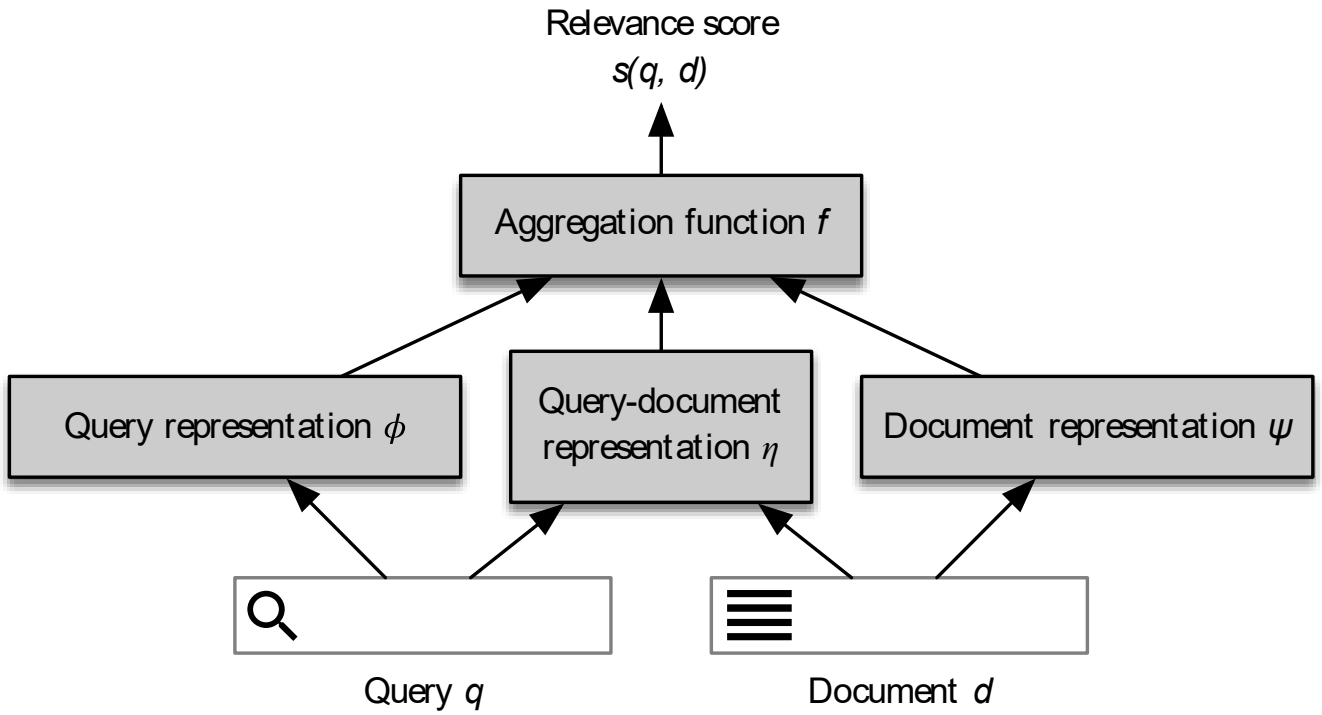
Any other instances of the framework?

- **Up to now**: queries and documents as multi-set of words
- **A lot of other potential signals to use**
  - Any idea?

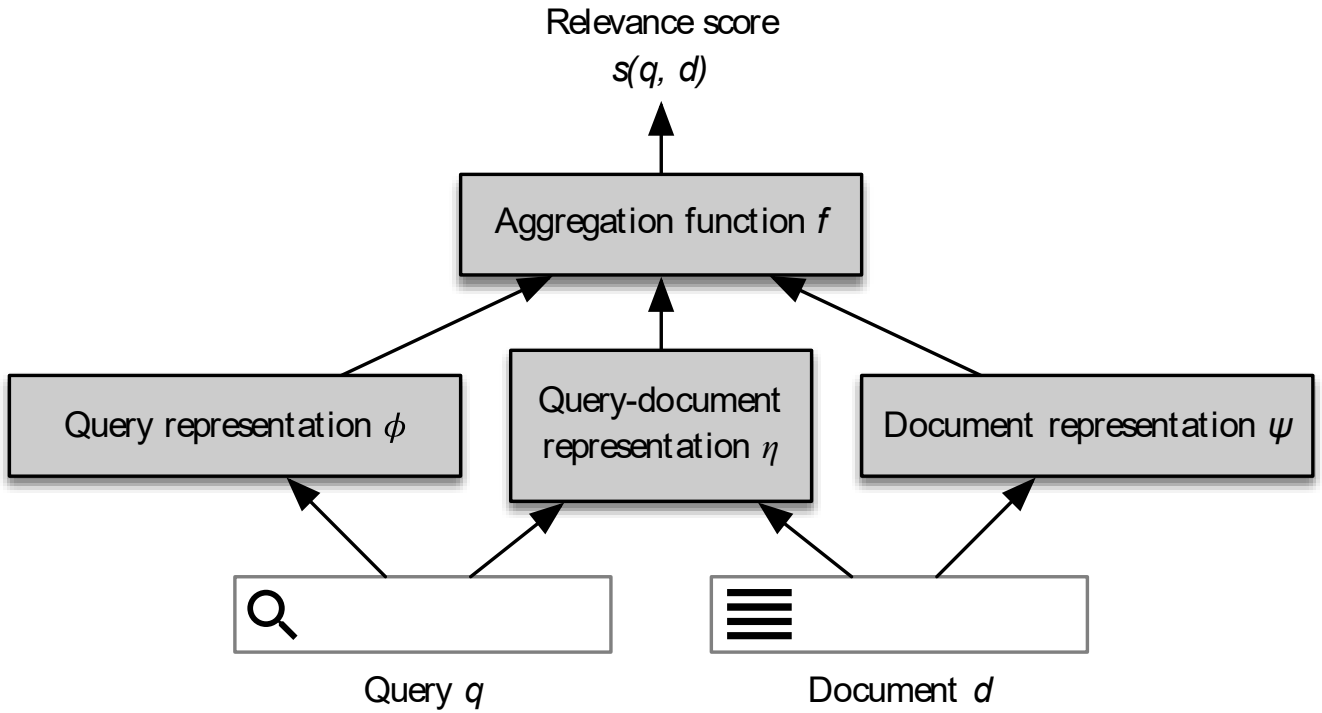
Any other instances of the framework?

- **Up to now**: queries and documents as multi-set of words
- **A lot of other potential signals to use**
  - Any idea?
- A document may have fields, it can be split into zones, it can be enriched with external text data (e.g., anchors)
- Additional information may be useful, e.g., In-Links, Out-Links, PageRank, # clicks, social links, etc.
- For queries: # terms, popularity in query logs, user profile info, etc.

General framework



Can we learn f?



What does it mean to learn f?

- We can use Machine Learning.
- Machine learning is *a field of computer science that uses statistical techniques to allow computer systems to **learn**, i.e., progressively improve performance on a specific task with data without being explicitly programmed.* (Wikipedia)
- What does it imply?
  - Existence of data
  - Optimization problem (task? measure?)

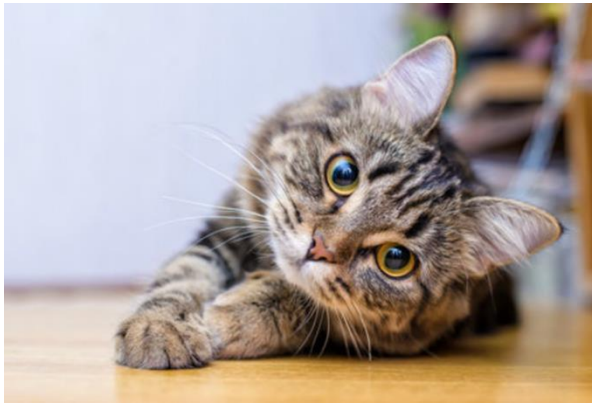
Machine Learning Tasks

Two broad categories, depending on whether there is a learning “signal” or “feedback” available to a learning system:

- **Supervised learning**: The computer is presented with example inputs and their desired outputs, given by a "teacher", and the goal is to learn a general rule that maps inputs to outputs.
  - For us, here: Classification / Regression problems
- **Unsupervised learning**: No labels are given to the learning algorithm, leaving it to find structure in its input.

Machine Learning Tasks

- **Supervised learning**: The computer is presented with example inputs and their desired outputs, given by a "teacher", and the goal is to learn a general rule that maps inputs to outputs. Classification below…



cat

dog

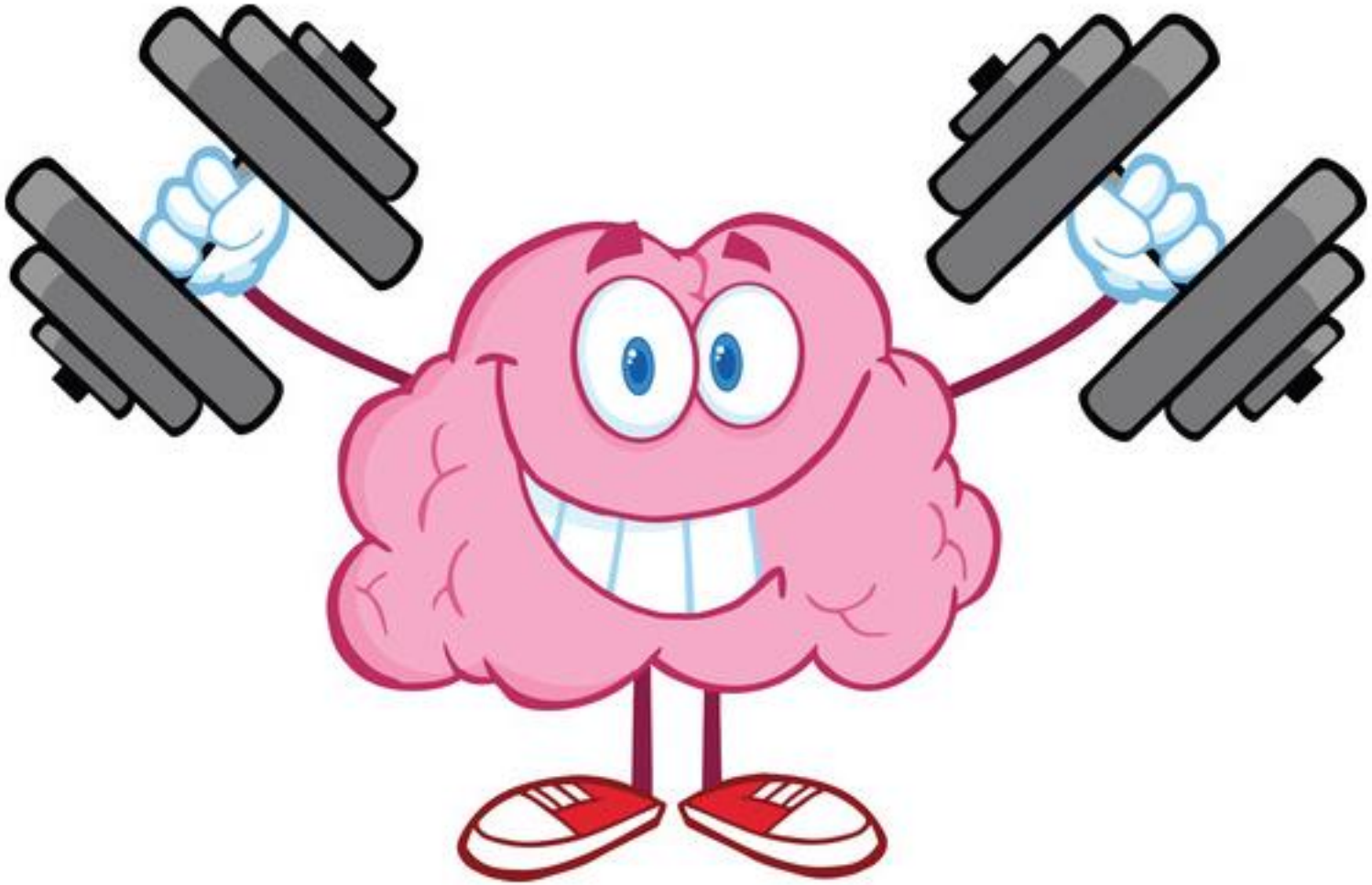
Useful Terminology

- Each item in the dataset is called **observation** or **instance**
- **Instances** of the dataset are described as a set of **features**, i.e., specific properties describing the observation.
- Each instance comes with a **label** (supervision)
- **Regression and Classification tasks** exploit the feature space and a label to learn a model that allows to: i) predict a specific real quantity (regression), ii) assign discrete labels to instances of data where the label is not known in advance

Useful Terminology

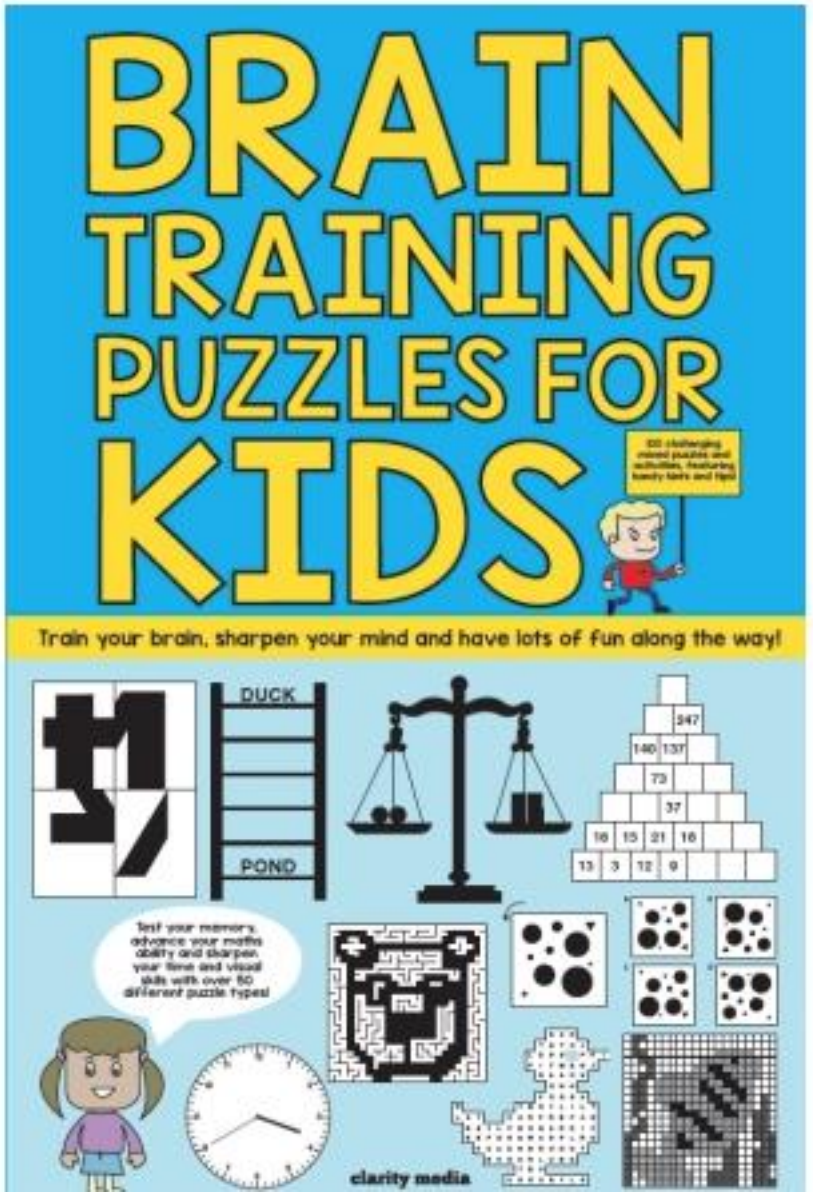
- Both **supervised** and **unsupervised** tasks are built around two distinct phases:
- **training** phase where the model is built. We refer to it as "to **fit** a model on a given set of data".
- **testing** phase where the model is used. We refer to it as "to **predict** class/label of a given set of data".
- The first phase (training) is done offline. The second phase (testing) is generally exploited online.

A parallelism…



A parallelism

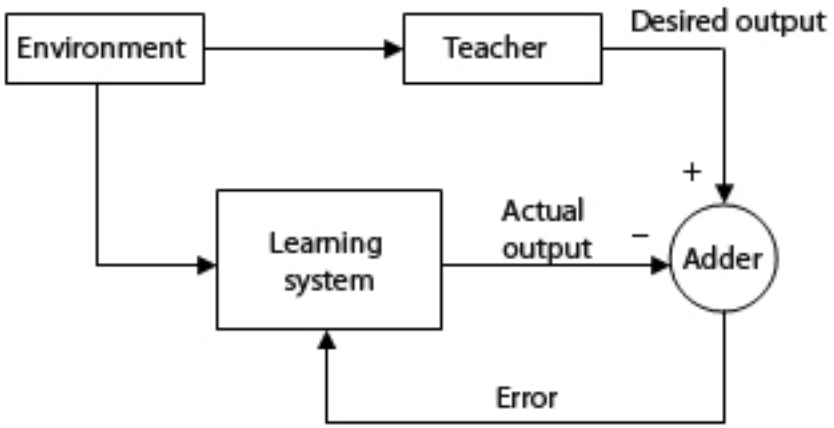
- For every puzzle, we have:
  - the puzzle itself
  - the solution to the puzzle
- Our “brain” produces a solution:
  - we compare our solution with the correct solution
  - by looking at how different the two are, we can fine-tune our brain
- … and eventually learn something
- Repeat **ad libitum**



General Framework

In machine-learning terms

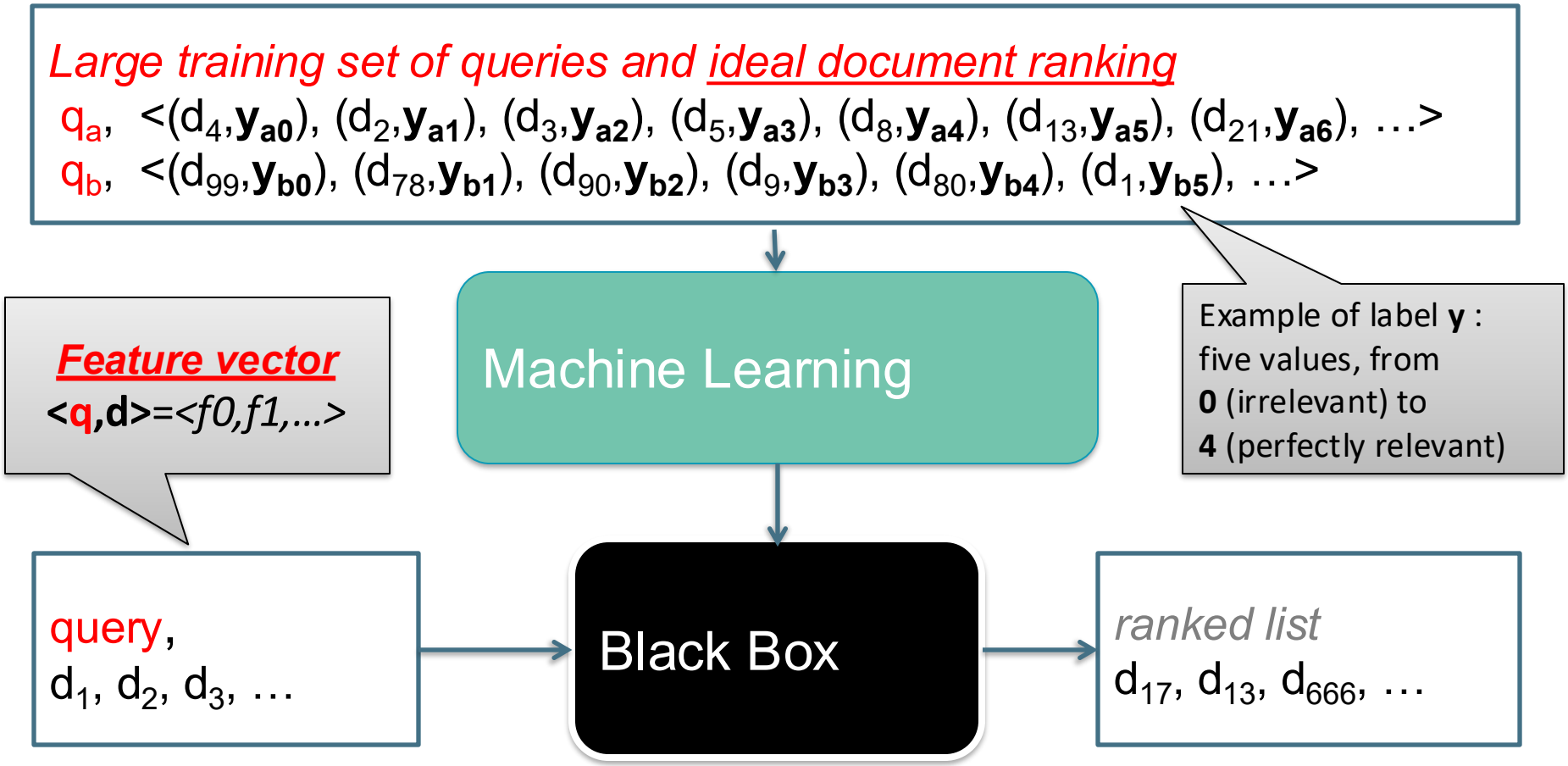
- a training instance is presented to the **machine learning algorithm**
- the algorithm produces an output or **prediction** for that instance
- the difference between the **prediction** and the desired output or **label** of the instance is called **loss** (or error)
- the loss is used to **update the model** of the machine learning algorithm
- repeat over **several training instances**, i.e., the **training dataset**
- repeat the whole process several times: **learning iterations** or **training epochs**



Learning to Rank (LtR)

- Goal?
- Let’s brainstorm together …

Learning to Rank (LtR)



- Even if the trained LtR models are regressors that score each candidate document, the goal is to guess the overall ranking, not the specific numerical labels

LtR features

- Modern systems – especially on the Web – use a great number of features associated with **<q,d>**=*<f0,f1,…>*

Log frequency of query word in anchor text
Query word in color on page
# of images on page
of (out) links on page
PageRank of page
URL length
URL contains character '~'
Page edit recency
Page length
BM25
query-url click count
url click count
url dwell time
….

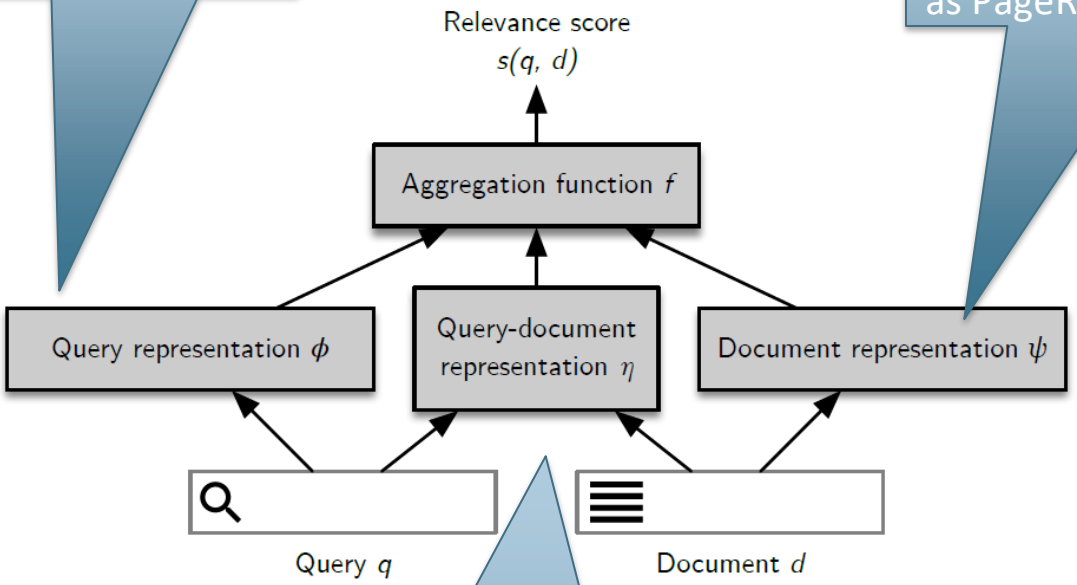
BM25 discussed later

….The *New York Times* (2008-06-03) quoted Amit Singhal as saying Google was using over 200 such features.

Features in LtR

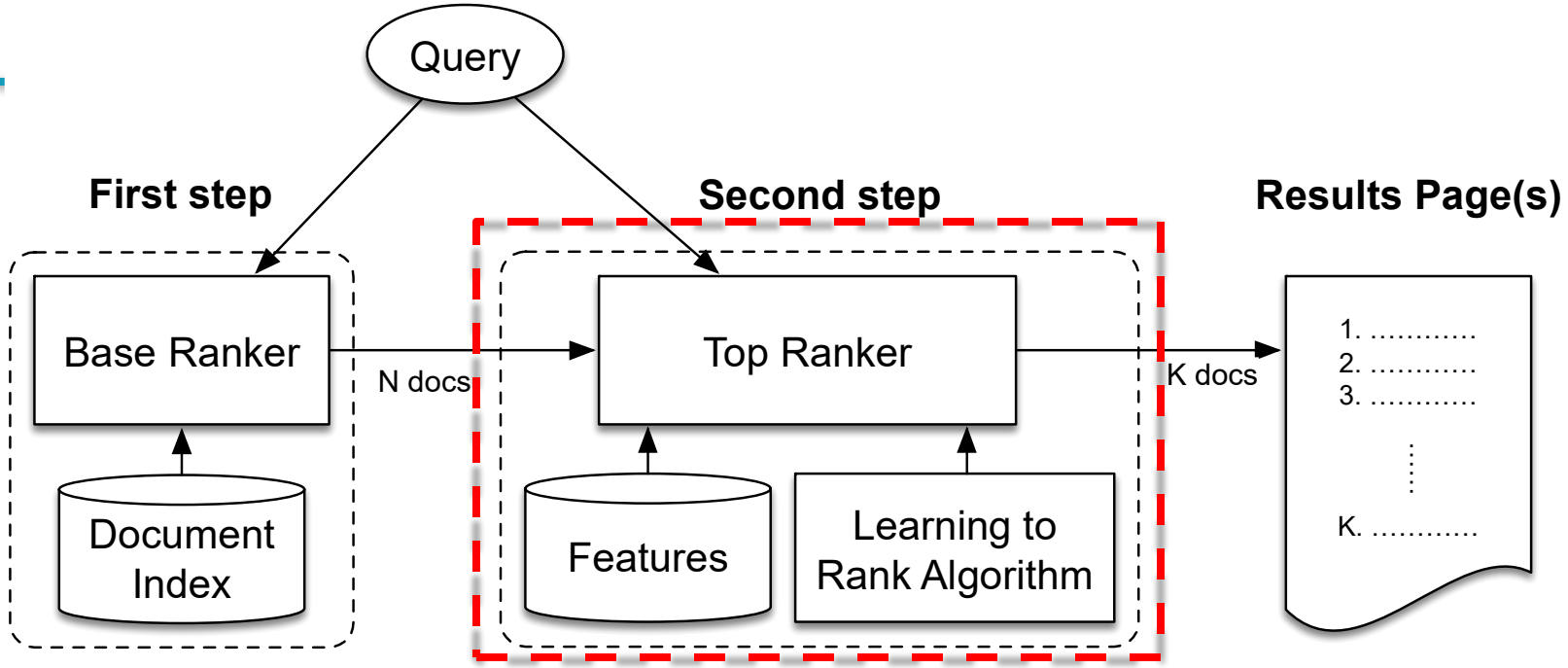
query-only features: same value for each document, such as query type, query length, …

query-independent features: document features with the same value for each query, such as PageRank, URL length, ..



query-dependent features: document features that depend on the query, such as query-url click count, query word in bold on page, BM25 …

Learning to Rank pipeline



- Budget considerations are very important for commercial WSEs
- The computational cost of LtR models must be in fact strictly accounted in the time budget available for processing queries in the incoming stream
  - Impact in the throughput of the system.
  - The IR community has started only recently to investigate low-level optimizations to reduce the execution time of some families of LtR rankers.

Example: Using classification for ad hoc IR

- Collect a training corpus of (*q, d, r*) triples
  - Relevance *r* is here binary (but may be multiclass, with 3–7 values)
  - Query-Document pairs are represented by feature vectors
    - **x** = (α, ω) α: cosine similarity, ω: min query window sz
      - ω is the shortest text span that includes all query words, regardless of the order
      - Query term proximity is a **very important** weighting factor
  - Train a machine learning model to predict the class *r* of a document-query pair

x = <q,d>= <f0,f1,…>

example	docID	query	cosine score	ω	judgment
Φ_1	37	linux operating system	0.032	3	*relevant*
Φ_2	37	penguin logo	0.02	4	*nonrelevant*
Φ_3	238	operating system	0.043	2	*relevant*
Φ_4	238	runtime environment	0.004	2	*nonrelevant*
Φ_5	1741	kernel layer	0.022	3	*relevant*
Φ_6	2094	device driver	0.03	2	*relevant*
Φ_7	3191	device driver	0.027	5	*nonrelevant*

Simple example: Using classification for ad hoc IR

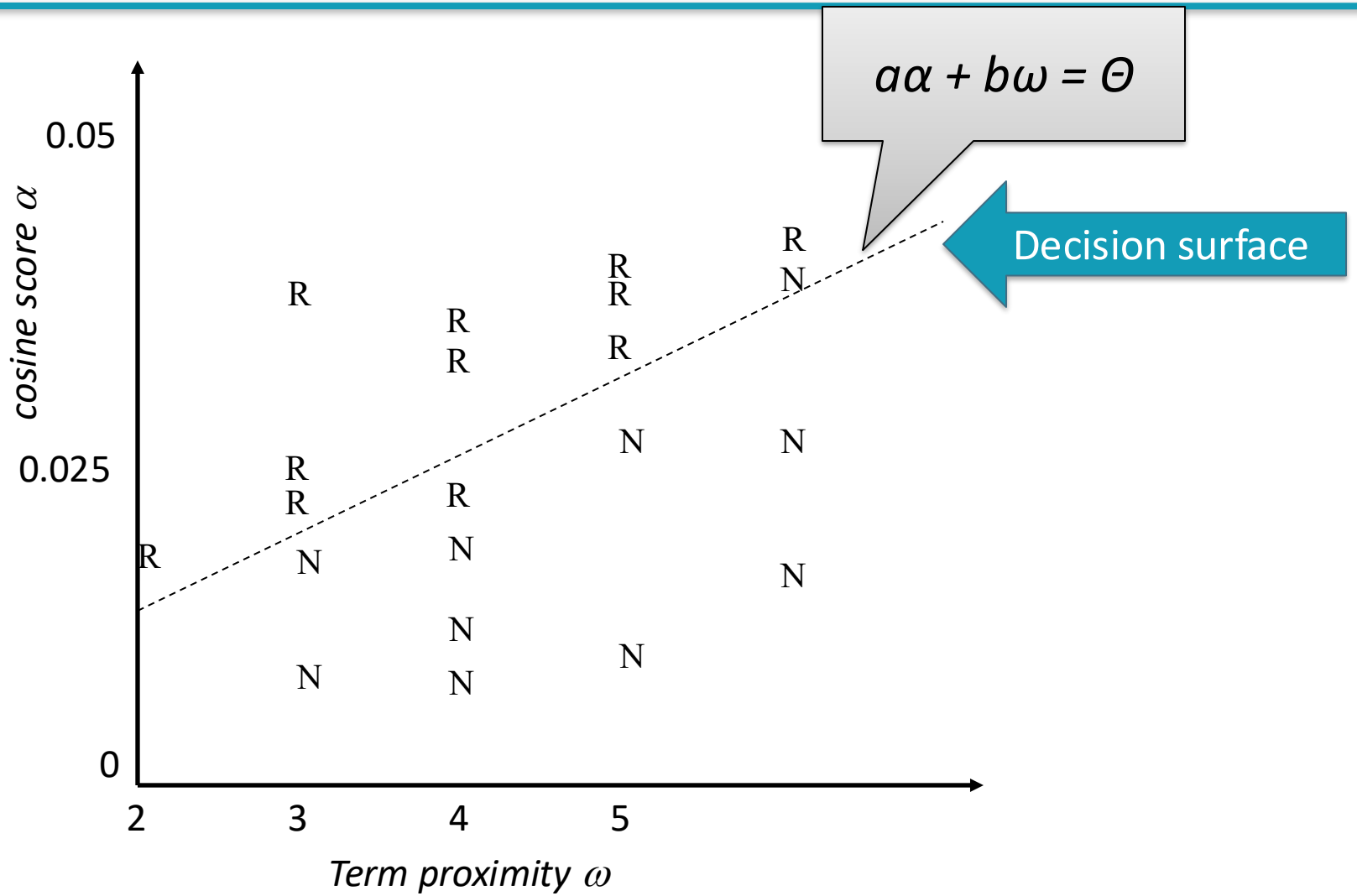
- A linear score function is then

$$Score(d, q) = Score(\langle \alpha, \omega \rangle) = a\alpha + b\omega$$

- The linear classifier must determine the parameters *a*, *b*, and *Θ* to:
  - decide *relevant* if $Score(d, q) > \Theta$
  - decide *irrelevant* if $Score(d, q) \leq \Theta$

- … just like text classification

Simple example: Using classification for ad hoc IR



Learning to Rank approaches

- ***Pointwise***
  - Each query-document pair is associated with a score
  - The objective is to predict such score
    - can be considered a pure *regression problem* or a *multiclass classification problem*
  - Does not consider the position of a document into the result list

Previous example: linear combination of α and ω

- ***Pairwise***
  - We are given pairwise preferences, d_1 is better than d_2 for query q
  - The objective is to predict a score that preserves such preferences
    - Can also be considered a *binary classification problem*
  - It doesn't consider the relevance of a document into the result list

RankNet (derivable loss function)

- ***Listwise***
  - We are given the ideal ranking of results for each query
  - Objective is to maximize the quality of the whole resulting ranked list by exploiting the whole list at training time
    - We need some improved approach … to maximize NDCG@K
    - Usually we cannot directly apply (Stochastic) Gradient Descent

BM25

- Okapi **BM25**
  - BM: best matching
  - Okapi is the name of an IR system first using this metric

$$\mathrm{BM25}(d, q) = \sum_t \mathrm{IDF}_t \, \tau(F_t)$$

- BM25 is a *probabilistic* ranking function, SOTA method better than cosine for BOW rankings
  - Using *term independence* assumption to approximate the document probability of being relevant
  - The model pay attention to *document length* besides *term frequency*

BM25

$IDF_t = \log(N/n_t)$ is the inverse document frequency

$$\mathrm{BM25}(d,q) = \sum_t \mathrm{IDF}_t \, \tau(F_t)$$

If $l_d = L$, then $F_t = TF$

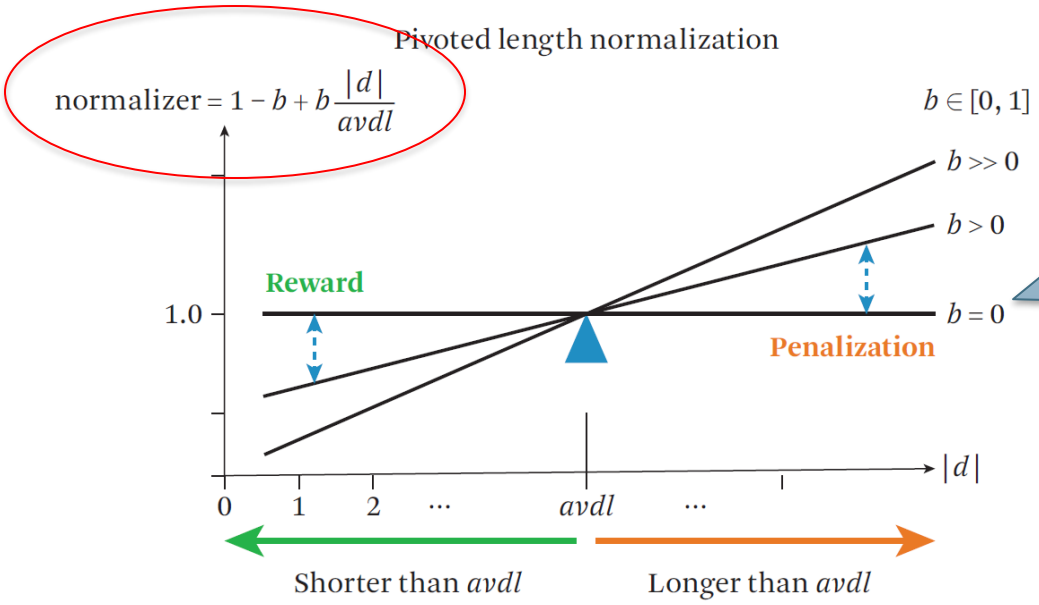
$$F_t = \frac{f_{t,d}}{1 - b + b \cdot l_d / L}$$

- $f_{t,d}$ is the frequency of term t in document d
- l_d is the length of document d
  - longer documents are less important
- L is the average document length in the collection
- b determines the importance of l_d

BM25

Pivoted transformation for Doc length, where ***L*** = *avdl (average doc length)*

$$F_t = \frac{f_{t,d}}{1 - b + b \cdot l_d / L}$$



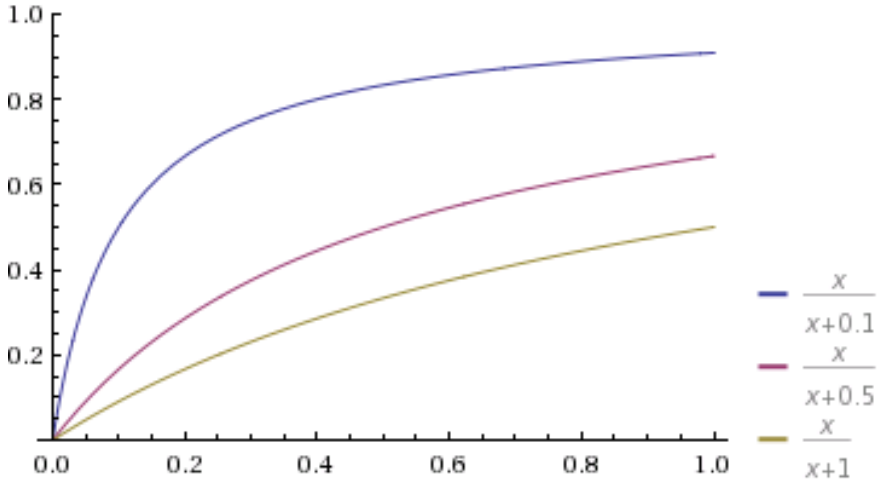
For b>0, the normalizer (denominator) becomes larger that 1.0

We aim to penalize long documents because they naturally have a better chance to match any query.

BM25

$$\mathrm{BM25}(d,q) = \sum_t \mathrm{IDF}_t \, \tau(F_t)$$

$$\tau(F_t) = \frac{F_t}{k + F_t}$$



- τ is a ***saturation function***, modeling *non-linearity* of term frequencies contribution, in turn normalized with document lenght

- Without tuning the two hyperparameters, k between 1.2 and 2, and b = 0.75.

BM25 and BM25F

- BM25 can be extended to handle *structured (multi-field) documents*
  - e.g., for a scholar paper: title, abstract, summary, author
  - e.g., for a web page: title, url, body, anchor(s)
- The extended function is named ***BM25F***:

$$\mathrm{BM25F}(d,q) = \sum_t \mathrm{IDF}_t \, \tau(F_t) \qquad F_t = \sum_s \frac{w_s \cdot f_{t,s}}{1 - b_s + b_s \cdot l_s / L_s}$$

  - w_s is a weight of field s
  - $f_{t,s}$ is the frequency of term t in field s (of document d)
  - l_s is the length of field s (of document d)
  - b_s determines the importance of l_s
  - L_s is the average length of field s in the collection

Need for tuning of BM25F

- BM25 has ***2*** free parameters:
  - *b,k*
- BM25F has ***2S+1*** free parameters (*S* is the no. of fields)
  - w_s,b_s,k
- How to find the best parameters of BM25(F) ?

- *Fit it into a learning-to-rank framework!*

LtR applied to BM25F

1. **Evaluation**
   - *Normalized Discounted Cumulative Gain @ 10*

$$DCG_p = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{log(1+i)}$$

relevance = 0,1,2,3,4 **2^{rel}-1** = 0,1,3,7,17

2. **Machine Learning Algorithm**
   - *Hypothesis space*: all the possible BM25F functions
   - *Optimization strategy*: try with *Gradient Descent over NDCG@K* ??
3. **The data**
   - We have a training instances *query/set of results*
     - Each query has a set of candidate results
     - Each results is *manually annotated with a relevance label (e.g., from 0 to 4 stars)*

LtR applied to BM25F

- Given a query q and a set of documents $D=\{d_1,d_2, \ldots\}$

- We want to learn a **model h** that allows to score and then rank the documents in D according to their relevance
  - Results = ***sort*** $\{h(d_1), h(d_2), \ldots\}$
  - where the function h is ***BM25F*** with a proper parameter set θ

- How to apply Gradient Descent ?
  - Unlike **pointwise** or **pairwise** approaches, the **listwise** approaches aim to directly optimize the evaluation metrics such as NDCG and MAP
  - The main difficulty in optimizing these evaluation metrics is that they are dependent on the **rank position** of documents induced by the scoring function, **not the numerical score** values output by the ranking function
  - We should compute the ***gradient*** of *sort* w.r.t. θ
    ….but sort is not a continuous and derivable function!

- Therefore, we cannot directly apply gradient descent

M. Taylor, H. Zaragoza, et al. *Optimisation Methods for Ranking Functions with Multiple Parameters.* **CIKM-06**

Wrap-up

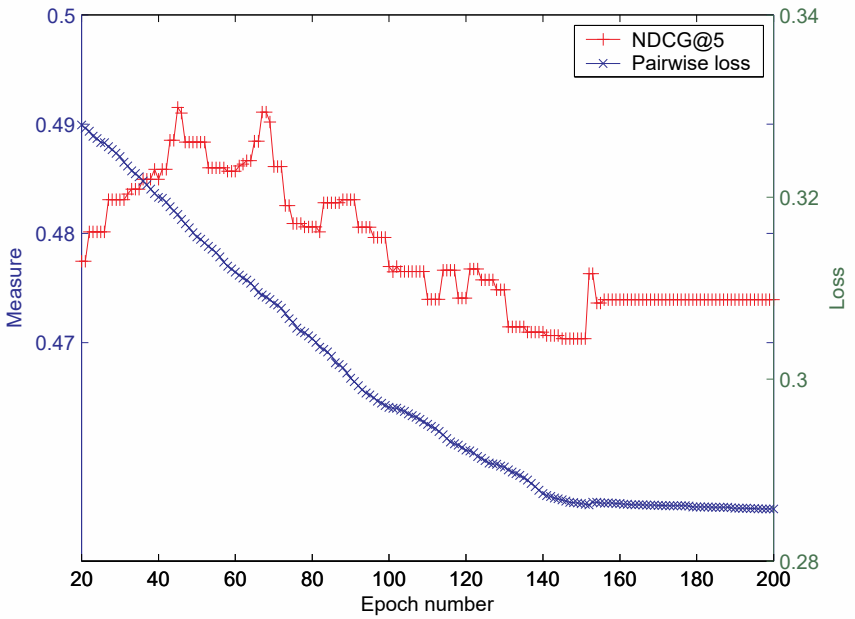
- NDCG@K is difficult to be optimized directly
- We found that it is possible to optimize a pairwise proxy measure, e.g., RankNet cost
  - Gradient Descent to tune BM25F
  - Minimizing a *pairwise loss*

- There exists much better ways to *approximate* the gradient of NDCG@K
  - **Lambda-MART** uses a loss for which it's possible to approximate a sort of NDGC gradients
  - **MART,** also known as **Gradient Boosted Regression Tree (GBRT)**, is an ensemble of regression trees
    - **See the library:** LightGBM

Issues of the pairwise approach

- We minimize the number of incorrectly classified pairs

- **But, to optimize NDCG, top result pairs are much more important than other pairs**



- In general, the number of document pairs violations, is **not** be a good indicator for NDCG